ALL SYSTEMS GO · 2026

~~Forget "podman generate systemd":~~

How Quadlet really works

Kritik Sachdeva (Technical Support Professional) IBM



ABOUT ME

Kritik Sachdeva

Technical Support Professional IBM

Work on Troubleshooting the storage / ceph / ODF

RHCA Level-11

Artist by passion

WHERE WE ARE GOING

01 What problem Quadlet solves

02 Groundwork: systemd, generators, Podman

03 What Quadlet is

04 Dependencies, on a real stack

05 Debugging when compilation fails

01

What problem Quadlet solves

The traditional approach

```
$ podman run -d --name myapp \
    -p 8080:80 nginx

$ podman generate systemd \
    --name myapp --files
```

- The unit is a snapshot of a container that is already running.

- Change one flag and you regenerate and reinstall the unit.

- The command you ran and the unit on disk drift apart.

- Maintaining the version control for systemd files is difficult

INTERLUDE

Change one flag, redo everything

Stop the container. Remove it. Run it again with the new flag. Regenerate the unit. Reinstall it. Reload systemd.



02

Groundwork: systemd, generators, Podman

Three pieces, one of them hidden



Podman

A daemonless container engine. No root-privileged process in the middle; regular users run containers themselves.

systemd

The service manager running as PID 1. It already knows how to start, order, supervise and restart things.

Generators

Small executables that turn config that systemd does not understand into transient unit files, before any unit is loaded.

Generators you already run every boot

`systemd-fstab-generator`	/etc/fstab becomes mount and swap units
`systemd-getty-generator`	Consoles found at boot become getty units
`systemd-cryptsetup-generator`	/etc/crypttab becomes device unlock units
`podman-user-generator`	.container, .volume, .network become services

Output is transient: written into tmpfs, rebuilt on every reload, gone on reboot.

03

What Quadlet is

Quadlet is a systemd generator for Podman

You describe a container in a small declarative file. At load time Quadlet compiles that file into a real systemd service, and systemd runs it like any other unit.



Two hooks, one per systemd manager

1 · SYSTEM MANAGER

`/usr/lib/systemd/system-generators/podman-system-generator`

Root containers, managed with systemctl.

Your files live in /etc/containers/systemd/

2 · USER MANAGER

`/usr/lib/systemd/user-generators/podman-user-generator`

Rootless containers, managed with systemctl –user.

Your files live in ~/.config/containers/systemd/

Everything from here on is rootless, so: the user hook.

Source and artifact

`*.container`

Source.

You write it. It goes in version control.

`*.service`

Artifact.

Rebuilt in tmpfs on every reload. Never edit it.

```
$ systemctl --user show web.service -p FragmentPath
FragmentPath=/run/user/1000/systemd/generator/web.service
```

What a .container file looks like

```
[Unit]
Description=Demo web server

[Container]
Image=nginx:alpine
PublishPort=8080:80

[Install]
WantedBy=default.target
```

Same INI shape as any systemd unit, saved with a Podman extension instead of `.service`.

`[Unit]` and `[Install]` are passed through to systemd unchanged.

`[Container]` is the Podman part, and that is the section Quadlet translates.

Four file types: .container, .volume, .network, .pod.

Every [Container] key is a flag

`Image=nginx:alpine`	→	`nginx:alpine`
`PublishPort=8080:80`	→	`-p 8080:80`
`Volume=webdata:/data`	→	`-v webdata:/data`
`Network=webnet.network`	→	`--network webnet`

INTERLUDE

Nothing is watching your files

Editing a .container changes nothing on its own. Quadlet only runs when systemd reloads.

```
$ systemctl --user
daemon-reload
```



04

Dependencies, on a real stack

Three files, one directory

~/.config/containers/systemd/

```
# webcontent.volume
[Volume]
VolumeName=webcontent
```

```
# webnet.network
[Network]
NetworkName=webnet
```

```
# webserver.container
[Container]
Image=nginx:alpine
Network=webnet.network
Volume=webcontent.volume:/usr/share/nginx/html
PublishPort=8080:80

[Install]
WantedBy=default.target
```

04 · DEMO

Start one unit, get three

```
$ systemctl --user daemon-reload
$ systemctl --user start webserver
$ curl -s localhost:8080 | head -1

$ systemctl --user \
   list-dependencies webserver
$ systemctl --user stop \
   webnet-network.service
```

Lab Demo

You never started the volume or the network. Requires= pulled them in, and propagated the stop back.

Two references, one graph



You wrote two lines. Quadlet wrote the graph.

Two axes, never one

`Requires=`

What must be there.

Pulls the unit in. If it fails, we do not start. Says nothing about order.

`After=`

When we may start.

Waits for it to finish starting. Does not pull it in at all.

```
REQUIRES= ONLY
t0 · postgres ▉ ▉ ▉ ▉ ready at 4s
t0 · myapp ▉ ▉ connects at 0.2s → crash
```

```
REQUIRES= AND AFTER=
t0 · postgres ▉ ▉ ▉ ▉ ready at 4s
          t4 · myapp ▉ ▉ connects → healthy
```

You need both. Quadlet emits both.

INTERLUDE

It works on my laptop. It just loses the race on boot.



You cannot enable a generated unit

```
$ systemctl --user enable webserver.service
Failed to enable unit: unit is transient or generated
```

Generated units live in tmpfs. There is nothing on disk to symlink into a `.wants/` directory.

```
[Install]
WantedBy=default.target
```

In the source file. Quadlet handles it at generation time, and it is the only route to boot behaviour.

05

Debugging when compilation fails

05 · DEMO

Breaking it on purpose

```
Network=does-not-exist.network
→ Failed: unit not found.

PublishPort=80:80
→ Job failed. See journal.
```

Lab Demo

Two failures that look alike from the outside. One never compiled; the other compiled and then died.

INTERLUDE

“Unit not found.” The file is right there.



Fault 1 — it never compiled

```
# webserver.container
Network=does-not-exist.network
```

```
$ systemctl --user start webserver
Failed: unit not found.
```

01	`systemctl --user status webserver`	No such unit. systemd never saw one.
02	`systemctl --user cat webserver`	Nothing to show. Compilation produced no file.
03	`journalctl -b -t quadlet-generator`	The generator already logged the error this boot.
04	`podman-user-generator --user --dryrun`	Re-run it by hand to see the failing key.
05	`fix source · daemon-reload`	The unit exists again.

Same shape for any compile-time fault: a typo, a bad Volume= reference, a subnet collision. The unit simply is not there.

Fault 2 — it compiled, then died

```
# webserver.container
PublishPort=80:80
```

```
$ systemctl --user start webserver
Job failed. See journal.
```

06	`systemctl --user status webserver`	The unit exists and failed. Compilation was fine.
07	`journalctl --user -u webserver -n 50`	cannot bind to port 80: permission denied
08	`podman ps -a · podman inspect`	Podman has the last word, not systemd.
09	`PublishPort=8080:80 · daemon-reload`	Rootless cannot bind below 1024.

Same shape for a failing healthcheck, a missing linger session, or an SELinux denial on a mount.

TAKEAWAYS

.container is source. .service is output.

References become systemd edges, not just Podman flags.

When it fails, debug the generator before the container.

Quadlet is not magic. It is a generator. Once you can see what it generates, you can reason about it.

ALL SYSTEMS GO · 2026

Thank you. Questions?

Kritik Sachdeva IBM